Recursion

Outline

Recursive algorithms

Tree of recursive calls and call stack

Correctness of recursive algorithms

Factorial

Fibonacci numbers

Greatest common divisor

Fast exponentiation

Towers of Hanoi

From iterative to recursive algorithms

- In the previous chapters every algorithm was **iterative**: a sequence of instructions and loops
- We counted its number of steps line by line (chapter «Running time») and we proved its correctness with loop invariants (chapter «Correctness»)

In this chapter we study a different way of designing algorithms:
an algorithm may **call itself**
on smaller instances of the same problem.

Plan of the chapter

- We define recursive algorithms, base cases and recursive steps, and we see why a recursive algorithm terminates
- We describe how a recursive algorithm is executed: the **tree of recursive calls** (which determines the number of steps) and the **call stack** (which determines the memory)
- We show that the correctness of a recursive algorithm is proved by **strong induction**
- We apply these tools to five classical recursive algorithms: the factorial, the Fibonacci numbers, the greatest common divisor, fast exponentiation and the Towers of Hanoi
- For each of them we prove the correctness, we count the number of steps by unrolling a recurrence, and we execute it step by step

Recursive algorithms

Introduction: Recursive algorithms

- In this section we define what a recursive algorithm is: base cases and recursive step
- We see why a recursive algorithm **terminates**, and what can go wrong when it does not
- Finally we fix how the number of steps of a recursive algorithm is counted: it leads to a **recurrence**

Self-similar problems

- Many problems have a natural **self-similar** structure: the solution of an instance can be easily obtained from the solution of one or more **smaller instances of the same problem**

- For example, for every natural number $n \geq 1$,

$$n! = n \cdot (n-1)!$$

  that is, the factorial of n is obtained from the factorial of the smaller number $n-1$

- Algorithms that exploit this structure are called recursive algorithms

Definition 1: recursive algorithm

*A **recursive algorithm** is an algorithm that solves an instance of a problem by **calling itself** on one or more **smaller instances** of the same problem, and then combining the results of these calls.*

The two parts of a recursive algorithm

Every recursive algorithm is made of two parts:

1. **Base case(s)**: one or more instances that are small enough to be solved directly, without any recursive call

   (for example, $0! = 1$)

2. **Recursive step**: on every other instance, the algorithm calls itself on smaller instances and combines the returned results

   (for example, $n! = n \cdot (n-1)!$)

Calls and size

- Each execution of the algorithm on a given input is called a **call** (or invocation)
- A call that is made from inside another call is a **recursive call**
- To measure how "small" an instance is, we associate to each input a **size**, which is a natural number
- For example the value n of the input number, or the number n of entries of an array $a[1..n]$

Termination

- A recursive algorithm terminates only if the chain of recursive calls eventually reaches a base case
- If the size of the instance strictly decreases at each recursive call, we obtain a strictly decreasing sequence of natural numbers

$$n_0 > n_1 > n_2 > \cdots$$

- There is no infinite strictly decreasing sequence of natural numbers (by the well-ordering property of $\mathbb{N}$): after at most n_0 calls the size cannot decrease anymore

Termination

Therefore, if every instance which is not a base case
is reduced to strictly smaller instances,

a base case is necessarily reached
after finitely many calls.

Key idea of recursion

- Solve the **base cases** directly
- Reduce every other instance to one or more **strictly smaller** instances of the same problem, solve them by recursive calls, and combine their results
- Since the size is a natural number that **strictly decreases** at every recursive call, a base case is always reached and the algorithm **terminates**

What happens if one of these ingredients is missing?
Let us see what can go wrong.

Example: what can go wrong (1/2)

Missing base case.

- Consider the function `WrongFactorial`(n) that returns $n \cdot$ `WrongFactorial`$(n-1)$, without any base case
- The call `WrongFactorial`(2) calls `WrongFactorial`(1), which calls `WrongFactorial`(0), which calls `WrongFactorial`(-1), and so on forever
- The recursion never stops

Example: what can go wrong (2/2)

Size not decreasing.

- A function that, on input n, calls itself on input n (or $n+1$) never reaches a base case $n = 0$ when $n \geq 1$

In practice.

- An infinite recursion is stopped by the computer when the memory reserved to the calls is exhausted
- This is a **stack overflow** error

Counting the steps of a recursive algorithm

- We use the same conventions as in the chapter «Running time»: each execution of a line of pseudo-code is **one step**
- A line that contains a recursive call counts as one step **plus** all the steps executed by that recursive call
- As a consequence, the number of steps $S(n)$ of a recursive algorithm is naturally expressed in terms of the number of steps on smaller instances, i.e., by a **recurrence**, such as

$$S(n) = S(n-1) + 2.$$

Solving the recurrences

In this chapter we solve the recurrences
of the step counts by **simple unrolling**
(also called **expansion**):

we replace repeatedly $S(\cdot)$ by its definition
until we reach a base case.

Summary

Recursive algorithms:

- call themselves on smaller instances of the same problem and combine the results
- base case(s), solved directly, and recursive step

Termination:

- the size is a natural number that strictly decreases at every recursive call
- without a base case, or with a size that does not decrease: infinite recursion (stack overflow)

Number of steps:

- described by a recurrence, solved by unrolling

Tree of recursive calls and call stack

Introduction: Tree of recursive calls and call stack

- In this section we describe how a recursive algorithm is actually executed
- The **tree of recursive calls** shows all the calls: its number of nodes determines the number of steps
- The **call stack** keeps the active calls in memory: its size is given by the **depth** of the tree
- These two pictures will be used for every algorithm of the chapter

Definition 2: tree of recursive calls

*The **tree of recursive calls** of a recursive algorithm on a given input is the rooted tree in which:*

- *every node is a call of the algorithm;*
- *the root is the initial call;*
- *the children of a node are the recursive calls made directly by that call, from left to right in the order in which they are executed.*

The leaves of the tree are the calls that do not make recursive calls (typically the base cases).

Nodes, steps and order of the calls

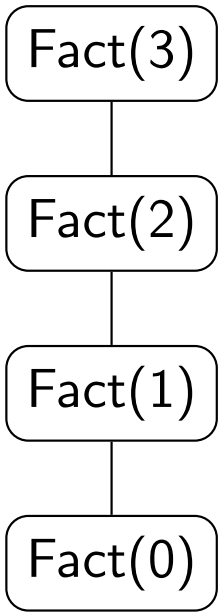
- The total number of calls executed by the algorithm is the **number of nodes** of the tree
- If every call executes a constant number of steps (excluding the steps of its recursive calls), then the number of steps of the algorithm is proportional to the number of nodes of the tree
- The calls are executed following a **depth-first** visit of the tree:
  - a call starts, executes its first recursive call (and the whole subtree below it) completely, then its second recursive call, and so on;
  - it returns its result only when all its recursive calls have returned.

Example: two shapes of trees of recursive calls (1/2)

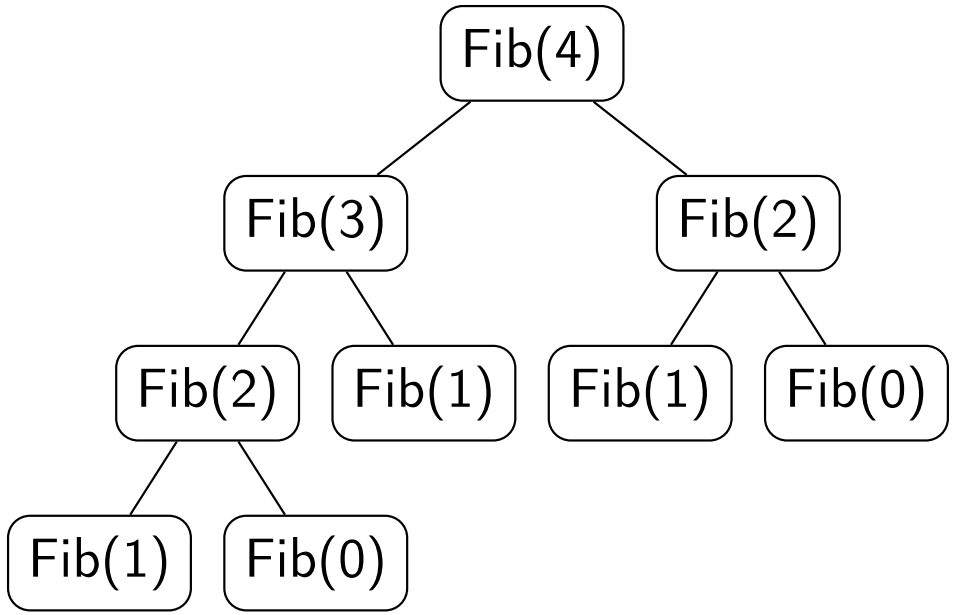
- If each call makes **one** recursive call (as for $n! = n \cdot (n-1)!$), the tree is a **path**
- If each call makes **two** recursive calls, the tree **branches** and the number of nodes can grow very quickly
- This happens for the Fibonacci numbers, defined by

$$F(n) = F(n-1) + F(n-2) \qquad \text{with } F(0) = 0 \text{ and } F(1) = 1$$

Example: two shapes of trees of recursive calls (2/2)



one call: a path



two calls: the tree branches

Stack frames

- When a call starts, the computer reserves a piece of memory, called a **stack frame** (or activation record)
- The frame contains:
  - the parameters of the call;
  - its local variables;
  - the point of the program where the execution must continue when the call returns.

The call stack

- The frames are stored in a memory area organised as a **stack** (the **call stack**):
  - when a call starts, its frame is **pushed** on top of the stack;
  - when a call returns, its frame is **popped** from the top of the stack, and the execution continues in the call whose frame is now on top.
- At any moment, the frames in the stack are exactly the calls on the path from the root of the tree of recursive calls to the call currently executed

Definition 3: depth of the recursion

*The **depth of the recursion** of a recursive algorithm on a given input is the maximum number of calls that are simultaneously active, i.e., the maximum number of nodes on a path from the root to a leaf of the tree of recursive calls.*

Memory used by the call stack

The maximum number of stack frames
simultaneously in memory
is equal to the **depth of the recursion**.

If every frame uses a constant amount of memory,
the memory used by the call stack is

Θ(depth of the recursion).

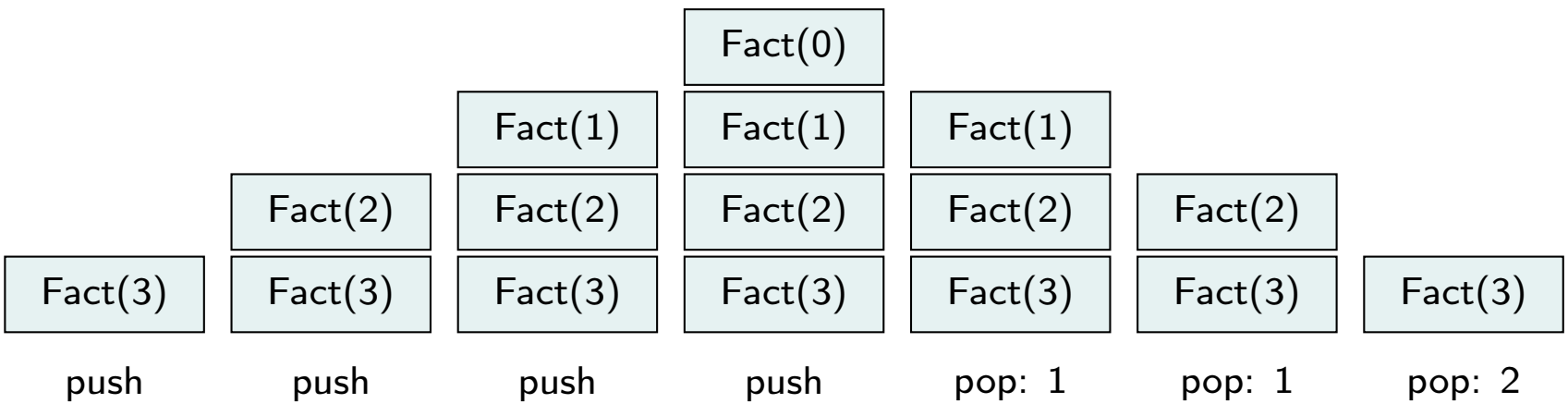
Steps versus memory

- Note the difference:
  - the **number of steps** depends on the **number of nodes** of the tree;
  - the **memory** used by the call stack depends only on its **depth**.

- For the tree of `Fib(4)` above:
  - there are 9 calls;
  - but at most 4 of them are simultaneously in the stack (for example `Fib(4)`, `Fib(3)`, `Fib(2)`, `Fib(1)`).

Example: evolution of the call stack for Fact(3) (1/2)

The figure shows the call stack (top of the stack is up) after each push and after each pop, during the computation of 3! with the recursion $n! = n \cdot (n-1)!$, $0! = 1$.



Example: evolution of the call stack for Fact(3) (2/2)

- Four frames are pushed: `Fact(3)`, `Fact(2)`, `Fact(1)`, `Fact(0)`
- `Fact(0)` is a base case: it returns 1 and its frame is popped
- Then `Fact(1)` returns $1 \cdot 1 = 1$, `Fact(2)` returns $2 \cdot 1 = 2$ and, finally, `Fact(3)` returns $3 \cdot 2 = 6$ and the stack becomes empty
- The depth of the recursion is 4

The size of the call stack is limited

The size of the call stack is limited
(typically a few megabytes).

A very deep recursion
(for example, millions of nested calls)
causes a **stack overflow**,
even if the algorithm is correct and terminates.

Summary

Tree of recursive calls:

- nodes = calls, children = recursive calls in order of execution
- number of nodes = number of calls; the calls follow a depth-first visit

Call stack:

- a frame is pushed when a call starts, popped when it returns
- the frames in the stack are the calls on the path from the root to the current call

Steps and memory:

- steps $\leftrightarrow$ number of nodes; memory $\leftrightarrow$ depth of the recursion

Correctness of recursive algorithms

Introduction: Correctness of recursive algorithms

- In this section we see how to prove that a recursive algorithm returns the correct output
- The tool is **strong induction** on the size of the input: the structure of a recursive algorithm mirrors the structure of an induction proof
- We also compare this approach with the loop invariants used for iterative algorithms

Induction on the size of the input

- The natural tool to prove the correctness of a recursive algorithm is **mathematical induction** on the size of the input
- A recursive call can be made on any smaller instance, not only on the instance of size $n - 1$
- For this reason we use **strong induction**

Strong induction

Let $P(n)$ be a statement depending on $n \in \mathbb{N}$, and let $n_0 \in \mathbb{N}$. If

1. **base case**: $P(n_0)$ is true, and
2. **inductive step**: for every $n > n_0$, if $P(k)$ is true for every k with $n_0 \leq k < n$ (**inductive hypothesis**), then $P(n)$ is true,

then $P(n)$ is true for every natural number $n \geq n_0$.

- It is sometimes convenient to prove more than one base case (for example $n = 0$ and $n = 1$), and then the inductive step only for n larger than all of them

Recursion mirrors induction

The structure of a recursive algorithm mirrors the structure of an induction proof:

- the **base cases** of the algorithm correspond to the **base cases** of the induction;
- in the **recursive step**, we may assume (inductive hypothesis) that the recursive calls, which are made on smaller inputs, return correct results;
- we only need to show that the results are **combined** correctly.

Observation 1: correctness of recursive algorithms

Consider a recursive algorithm in which every input has a size in $\mathbb{N}$. Assume that:

1. *on every input which is a base case, the algorithm returns the correct output;*
2. *on every other input of size n, the algorithm makes a finite number of recursive calls, all on inputs of size strictly smaller than n, and returns the correct output whenever all these recursive calls return the correct output.*

Then the algorithm terminates and returns the correct output on every input.

Proof (1/2)

Let $P(n)$ be the statement:

"on every input of size n the algorithm terminates and returns the correct output".

We prove $P(n)$ for every $n \in \mathbb{N}$ by strong induction.

- Let $n \in \mathbb{N}$ and assume that $P(k)$ is true for every $k < n$

  (for $n = 0$ this assumption is empty)

- Consider an input of size n

Proof (2/2)

- If the input is a base case, then the algorithm terminates and returns the correct output by hypothesis 1
- Otherwise, by hypothesis 2, the algorithm makes a finite number of recursive calls on inputs of size $k < n$
  - by the inductive hypothesis, each of these calls terminates and returns the correct output;
  - hence the algorithm terminates and, by hypothesis 2, it returns the correct output.

Therefore $P(n)$ is true, and by strong induction $P(n)$ is true for every $n \in \mathbb{N}$. □

Link with loop invariants

- In the chapter «Correctness» we proved the correctness of iterative algorithms by means of **loop invariants**, which also have the structure of an induction proof
- The three properties of a loop invariant correspond to the parts of the proof by induction of a recursive algorithm

Loop invariants and induction

Loop invariant (iterative algorithm)	**Induction** (recursive algorithm)
Initialization: true before the first iteration	Base case(s) of the recursion
Maintenance: preserved by each iteration	Inductive step: correct combination of the results
Termination: the loop stops	The size strictly decreases towards a base case

Two directions

The main difference is the direction.

An iterative algorithm builds the solution "from the bottom"
(from the initialization towards the final iteration).

A recursive algorithm starts "from the top"
(the original instance) and reduces it to smaller instances
until a base case is reached.

Summary

Strong induction:

- the inductive hypothesis holds for every smaller size, not only for $n - 1$
- possibly with more than one base case

Correctness of a recursive algorithm:

- correct output on the base cases
- finitely many recursive calls on strictly smaller inputs, whose correct results are combined correctly

Loop invariants:

- initialization $\leftrightarrow$ base cases, maintenance $\leftrightarrow$ inductive step, termination $\leftrightarrow$ decreasing size

Factorial

Introduction: Factorial

- In this section we write the simplest recursive algorithm: the computation of $n!$
- We draw its tree of recursive calls, prove its correctness by induction and count its steps by unrolling the recurrence $S(n) = S(n-1) + 2$
- We also express its running time as a function of the input size, and execute it step by step

The problem

Given a natural number n, we consider the problem of computing the **factorial** of n:

$$n! = 1 \cdot 2 \cdot \ldots \cdot n \qquad \text{with } 0! = 1.$$

- Since $n! = n \cdot (n-1)!$ for every $n \geq 1$, the factorial has the following recursive definition:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot (n-1)! & \text{if } n \geq 1 \end{cases}$$

This is the pseudo-code of the recursive algorithm `Factorial`.

Algorithm: Factorial

Input : A natural number n
Output : The factorial $n!$

```
1 function Factorial(n)
2     if n = 0 then
3         return 1                          // base case
4     end
5     return n · Factorial(n − 1)           // recursive step
6 end
```

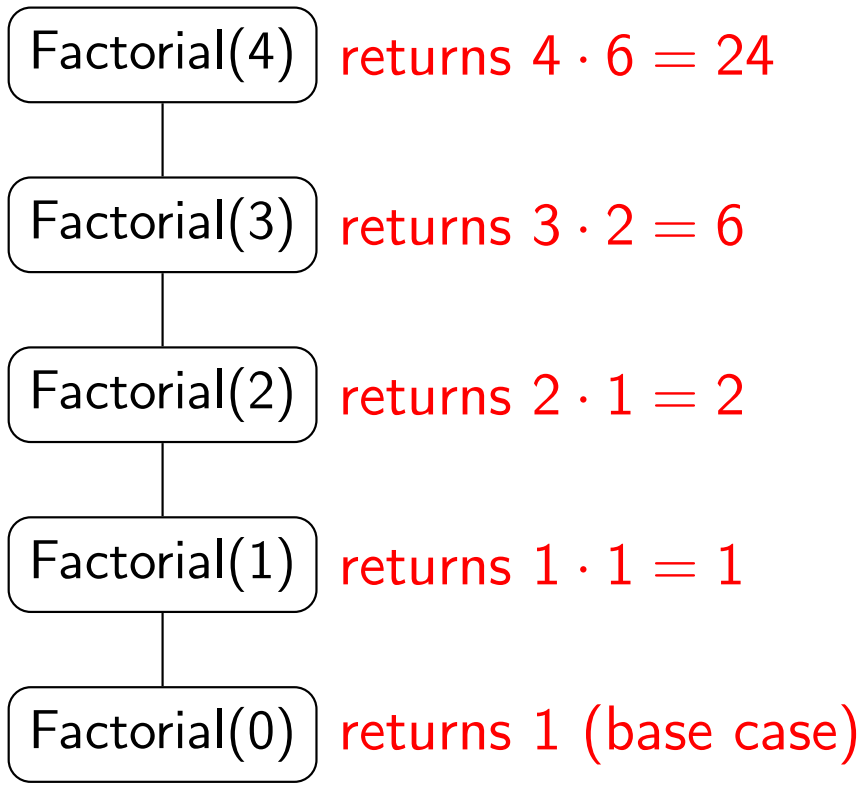
Run in Colab: Python · C++ · Java

How Factorial works

- The base case is $n = 0$, for which the algorithm returns $0! = 1$
- For $n \geq 1$, the algorithm calls itself on the smaller input $n - 1$ and multiplies the result by n
- The size of the input is the value n, which decreases by one at each recursive call: the base case is reached after n recursive calls
- In a C++ implementation with the type `unsigned long long` (64 bits), the result is correct only for $n \leq 20$, since $21! > 2^{64} - 1$

Example: tree of recursive calls of Factorial(4)

The tree is a path with 5 nodes. The value returned by each call is written in red next to the call.



The calls are made from the top to the bottom; the results are returned from the bottom to the top. The depth of the recursion is 5.

Observation 2: correctness of Factorial

For every natural number n, the call `Factorial`(n) *terminates and returns* $n!$.

Proof

We proceed by induction on n.

- **Base case** ($n = 0$): the condition of line 2 is true and the algorithm returns $1 = 0!$ at line 3
- **Inductive step** ($n \geq 1$): assume that $\texttt{Factorial}(n-1)$ terminates and returns $(n-1)!$ (inductive hypothesis). Since $n \neq 0$, the algorithm executes line 5 and returns

$$n \cdot \texttt{Factorial}(n-1) = n \cdot (n-1)! = n!.$$

Therefore $\texttt{Factorial}(n)$ returns $n!$ for every $n \in \mathbb{N}$. □

Number of steps: the recurrence

- Let $S(n)$ denote the number of steps executed by `Factorial` on the input n (there is only one input for each value of n, so this is also the worst case)
- If $n = 0$, lines 2 and 3 are executed once each:

$$S(0) = 2.$$

- If $n \geq 1$, lines 2 and 5 are executed once each, and line 5 also executes all the steps of the call `Factorial`$(n-1)$:

$$S(n) = S(n-1) + 2.$$

Number of steps: unrolling the recurrence

We solve the recurrence by unrolling it:

$$\begin{aligned} S(n) &= S(n-1) + 2 \\ &= \big(S(n-2) + 2\big) + 2 = S(n-2) + 2 \cdot 2 \\ &= \big(S(n-3) + 2\big) + 2 \cdot 2 = S(n-3) + 3 \cdot 2 \\ &\vdots \\ &= S(n-k) + k \cdot 2 \qquad \forall k \in \{1, 2, \ldots, n\}. \end{aligned}$$

Choosing $k = n$ (base case), we obtain

$$S(n) = S(0) + 2n = 2n + 2.$$

Number of steps and depth

- Therefore,

$$S(n) = 2n + 2 = \Theta(n),$$

since

$$\lim_{n\to\infty} \frac{2n+2}{n} = \lim_{n\to\infty} \left(2 + \frac{2}{n}\right) = 2 \in \mathbb{R}_{>0}.$$

- The depth of the recursion is $n + 1$ (the calls with inputs $n, n-1, \ldots, 0$ are all simultaneously in the stack), so the call stack uses $\Theta(n)$ memory

- The iterative algorithm that computes $n!$ with a `while` loop uses only a constant amount of memory

We now express the number of steps
as a function of the input size $|I|$.

Steps as a function of the input size (1/2)

- As in the chapter «Running time», we count each multiplication as one step
- The input is the natural number n, which requires $\lfloor \log_2 n \rfloor + 1$ bits (for $n \geq 1$), hence

$$|I| = \Theta(\log_2 n).$$

- This means that there exist constants $c_1, c_2 > 0$ such that, for all sufficiently large n,

$$c_1 \log_2 n \leq |I| \leq c_2 \log_2 n \qquad \Longleftrightarrow \qquad 2^{\frac{1}{c_2}|I|} \leq n \leq 2^{\frac{1}{c_1}|I|}.$$

Steps as a function of the input size (2/2)

- Since $S(n) = \Theta(n)$, there exist constants $a_1, a_2 > 0$ such that

$$a_1 n \leq S(n) \leq a_2 n$$

for all sufficiently large n

- Therefore

$$a_1 2^{\frac{1}{c_2}|I|} \leq S(n) \leq a_2 2^{\frac{1}{c_1}|I|},$$

that is,

$$T(|I|) = 2^{\Theta(|I|)}.$$

Running time of Factorial

- The number of steps of the `Factorial` algorithm is

$$S(n) = 2n + 2 = \Theta(n),$$

  and its depth of recursion is $n + 1$

- As a function of the input size $|I| = \Theta(\log_2 n)$, the worst-case running time is

$$T(|I|) = 2^{\Theta(|I|)},$$

  i.e., it is **linear** in the numerical value n,
  but **exponential** in the input size $|I|$.

Example: execution of Factorial(4) (1/2)

Calls (the stack grows)

1. `Factorial(4)`: $4 \neq 0 \Rightarrow$ it must compute $4 \cdot$ `Factorial(3)` and waits. stack: [4]
2. `Factorial(3)`: $3 \neq 0 \Rightarrow$ it must compute $3 \cdot$ `Factorial(2)` and waits. stack: [4, 3]
3. `Factorial(2)`: $2 \neq 0 \Rightarrow$ it must compute $2 \cdot$ `Factorial(1)` and waits. stack: [4, 3, 2]
4. `Factorial(1)`: $1 \neq 0 \Rightarrow$ it must compute $1 \cdot$ `Factorial(0)` and waits. stack: [4, 3, 2, 1]
5. `Factorial(0)`: $n = 0 \Rightarrow$ base case. stack: [4, 3, 2, 1, 0]

Example: execution of Factorial(4) (2/2)

Returns (the stack shrinks)

1. `Factorial(0)` returns 1. stack: [4, 3, 2, 1]
2. `Factorial(1)` returns $1 \cdot 1 = 1$. stack: [4, 3, 2]
3. `Factorial(2)` returns $2 \cdot 1 = 2$. stack: [4, 3]
4. `Factorial(3)` returns $3 \cdot 2 = 6$. stack: [4]
5. `Factorial(4)` returns $4 \cdot 6 = 24$. stack: [ ]

The algorithm terminates with result $24 = 4!$, after 5 calls.

Summary

The algorithm:

- base case $n = 0$, recursive step $n \cdot \texttt{Factorial}(n-1)$
- the tree of recursive calls is a path with $n+1$ nodes

Correctness:

- by induction on n

Number of steps:

- $S(n) = S(n-1) + 2$, hence $S(n) = 2n + 2 = \Theta(n)$; depth $n+1$
- $T(|I|) = 2^{\Theta(|I|)}$: exponential in the input size

Fibonacci numbers

Introduction: Fibonacci numbers

- In this section the recursive algorithm makes **two** recursive calls, and its tree of recursive calls branches
- We prove that the number of calls grows **exponentially**, because the same subproblems are solved again and again
- We compare the naive recursive algorithm with an iterative algorithm that executes only a linear number of steps

The problem

Given a natural number n, we consider the problem of computing the n-th **Fibonacci number** $F(n)$, defined as

$$F(n) = \begin{cases} n & \text{if } n \in \{0, 1\} \\ F(n-1) + F(n-2) & \text{if } n \geq 2 \end{cases}$$

The first Fibonacci numbers are:

n	0	1	2	3	4	5	6	7	8	9	10
$F(n)$	0	1	1	2	3	5	8	13	21	34	55

The definition is itself recursive, and it directly gives the following (naive) recursive algorithm `Fib`, with **two** base cases and **two** recursive calls.

Algorithm: Fib

Input : A natural number n
Output : The Fibonacci number $F(n)$

```
1 function Fib(n)
2 |   if n ≤ 1 then
3 |   |   return n                                  // base cases
4 |   end
5 |   return Fib(n − 1) + Fib(n − 2)      // recursive step
6 end
```

Run in Colab: Python · C++ · Java

Observation 3: correctness of Fib

For every natural number n, the call `Fib`(n) *terminates and returns* $F(n)$.

Proof

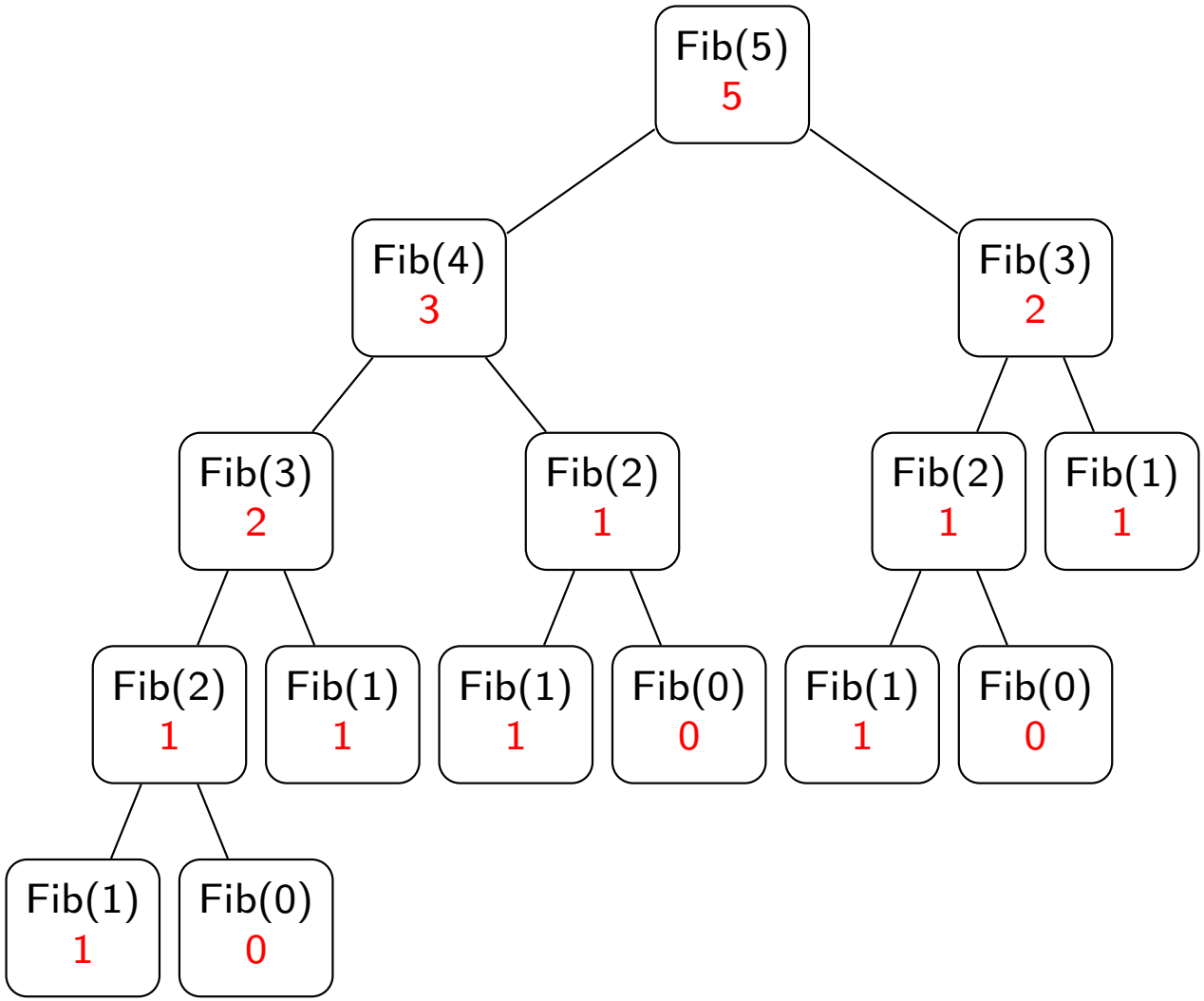
We proceed by strong induction on n.

- **Base cases** ($n \in \{0, 1\}$): the algorithm returns $n = F(n)$ at line 3
- **Inductive step** ($n \geq 2$): by the inductive hypothesis the two calls on the smaller inputs $n - 1$ and $n - 2$ return $F(n - 1)$ and $F(n - 2)$, and line 5 returns their sum $F(n)$

Here two base cases are needed, because the inductive step for n uses both $n - 1$ and $n - 2$. □

Example: tree of recursive calls of Fib(5) (1/2)

Each node is a call; the value returned by the call is written in red below it.



Example: tree of recursive calls of Fib(5) (2/2)

- The tree has 15 nodes: the algorithm makes 15 calls to compute $F(5) = 5$
- The same subproblems are solved many times:
  - `Fib(3)` is computed 2 times;
  - `Fib(2)` 3 times;
  - `Fib(1)` 5 times;
  - `Fib(0)` 3 times.
- The depth of the recursion is 5 (for example, the path `Fib(5)`, `Fib(4)`, `Fib(3)`, `Fib(2)`, `Fib(1)`)

Number of calls

- Let $C(n)$ denote the number of calls (nodes of the tree) executed by `Fib`(n)
- Each call with $n \geq 2$ generates one call on $n-1$ and one call on $n-2$, hence

$$C(n) = \begin{cases} 1 & \text{if } n \in \{0, 1\} \\ C(n-1) + C(n-2) + 1 & \text{if } n \geq 2 \end{cases}$$

- The first values are

$$C(0) = 1, \quad C(1) = 1, \quad C(2) = 3, \quad C(3) = 5,$$
$$C(4) = 9, \quad C(5) = 15, \quad C(6) = 25$$

- They are related to the Fibonacci numbers themselves

Observation 4: number of calls of Fib

For every natural number n,

$$C(n) = 2F(n+1) - 1.$$

Proof

We proceed by strong induction on n.

- **Base cases**: $C(0) = 1 = 2 \cdot 1 - 1 = 2F(1) - 1$ and $C(1) = 1 = 2 \cdot 1 - 1 = 2F(2) - 1$
- **Inductive step** ($n \geq 2$): assume $C(k) = 2F(k+1) - 1$ for every $k < n$. Then

$$\begin{aligned} C(n) &= C(n-1) + C(n-2) + 1 \\ &= \big(2F(n) - 1\big) + \big(2F(n-1) - 1\big) + 1 \\ &= 2\big(F(n) + F(n-1)\big) - 1 = 2F(n+1) - 1. \end{aligned}$$

□

Number of steps

- For example, $C(5) = 2F(6) - 1 = 2 \cdot 8 - 1 = 15$, as in the tree of `Fib(5)`
- Each call executes exactly 2 lines (lines 2–3 for a base case, lines 2 and 5 otherwise), hence the number of steps $S(n)$ satisfies

  $$S(n) = S(n-1)+S(n-2)+2 \quad (n \geq 2), \qquad S(0) = S(1) = 2,$$

  and therefore

  $$S(n) = 2C(n) = 4F(n+1) - 2.$$

- The number of calls grows **exponentially** in n

Observation 5: exponential number of calls of Fib

For every natural number $n \geq 2$,

$$2^{n/2} \leq C(n) \leq 2^n.$$

Proof (1/2)

Lower bound. We proceed by strong induction on $n \geq 2$.

- **Base cases**: $C(2) = 3 \geq 2 = 2^{2/2}$ and $C(3) = 5 \geq 2\sqrt{2} = 2^{3/2}$
- **Inductive step** ($n \geq 4$): assume the claim for every k with $2 \leq k < n$; in particular for $k = n - 2 \geq 2$
- Since $C(n-1) \geq C(n-2)$ (the sequence C is non-decreasing),

$$C(n) = C(n-1) + C(n-2) + 1 > 2\,C(n-2) \geq 2 \cdot 2^{(n-2)/2} = 2^{n/2}.$$

Proof (2/2)

Upper bound. Also by strong induction:

- $C(2) = 3 \leq 4$ and $C(3) = 5 \leq 8$
- for $n \geq 4$,

  $$C(n) = C(n-1) + C(n-2) + 1 \leq 2^{n-1} + 2^{n-2} + 2^{n-2} = 2^n,$$

  where we used $1 \leq 2^{n-2}$.

□

The naive algorithm is exponential

- Note that $C(1) = 1 < 2^{1/2}$: this is why the lower bound is stated for $n \geq 2$
- Therefore $S(n) = 2C(n) = 2^{\Theta(n)}$: the naive recursive algorithm is **exponential** in n
- For example, the computation of $F(30) = 832040$ requires

$$C(30) = 2F(31) - 1 = 2 \cdot 1346269 - 1 = 2692537 \text{ calls}$$

- (It can be shown that $F(n)$ grows as $\varphi^n/\sqrt{5}$, where $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$ is the golden ratio; hence $C(n) = \Theta(\varphi^n)$.)

An iterative algorithm

The reason of the exponential number of calls
is that the naive algorithm **solves the same subproblems again and again** (see the tree of `Fib(5)`).

An iterative algorithm computes each Fibonacci number only once,
from $F(0)$ and $F(1)$ upwards,
keeping only the last two values.

Algorithm: IterativeFib (1/2)

Input : A natural number n
Output : The Fibonacci number $F(n)$

```
1 function IterativeFib(n)
2     if n ≤ 1 then
3         return n
4     end
5     prev := 0
6     curr := 1
7     i := 2
```

Algorithm: IterativeFib (2/2)

```
function IterativeFib(n) (continued)
8       while i ≤ n do
9           next := prev + curr
10          prev ← curr
11          curr ← next
12          i ← i + 1
13      end
14      return curr
15  end
```

Run in Colab: Python · C++ · Java

Number of steps of IterativeFib

- For $n \geq 2$:
  - lines 2, 5, 6, 7, 14 are executed once;
  - line 8 is executed n times;
  - lines 9–12 are executed $n - 1$ times.
- Hence

$$S(n) = 5 + n + 4(n-1) = 5n + 1 = \Theta(n).$$

- (Loop invariant: before each test of line 8, $prev = F(i-2)$ and $curr = F(i-1)$; at the end $i = n+1$ and $curr = F(n)$.)

Recursive versus iterative

n	$F(n)$	steps of `Fib`: $4F(n+1)-2$	steps of `IterativeFib`: $5n+1$
5	5	30	26
10	55	354	51
20	6765	43782	101
30	832040	5385074	151

- The naive recursive algorithm `Fib` executes $S(n) = 4F(n+1) - 2 = 2^{\Theta(n)}$ steps (exponential in n)
- `IterativeFib` executes $S(n) = 5n + 1 = \Theta(n)$ steps (linear in n)

As functions of the input size

Since $|I| = \Theta(\log_2 n)$,
as functions of the input size:

the naive algorithm is even **doubly exponential**,

while the iterative one is **exponential**
(as `Factorial`).

Repeated subproblems

- The recursion itself is not the problem
- The problem is the **repetition of identical subproblems**
- Storing the solution of each subproblem the first time it is solved, and reusing it afterwards, is the key idea of the technique called **Dynamic Programming**

Example: execution for $n = 4$ (1/3)

Execution of `Fib(4)`: the calls follow the depth-first order of the tree of `Fib(4)` seen above (indentation = depth in the stack).

call	action
`Fib(4)`	calls `Fib(3)`
`  Fib(3)`	calls `Fib(2)`
`    Fib(2)`	calls `Fib(1)`
`      Fib(1)`	base case, returns 1
`    Fib(2)`	calls `Fib(0)`
`      Fib(0)`	base case, returns 0
`    Fib(2)`	returns $1 + 0 = 1$
`  Fib(3)`	calls `Fib(1)`
`    Fib(1)`	base case, returns 1
`  Fib(3)`	returns $1 + 1 = 2$

Example: execution for $n = 4$ (2/3)

The execution continues with the second recursive call of `Fib(4)`.

call	action
`Fib(4)`	calls `Fib(2)`
`  Fib(2)`	calls `Fib(1)`
`    Fib(1)`	base case, returns 1
`  Fib(2)`	calls `Fib(0)`
`    Fib(0)`	base case, returns 0
`  Fib(2)`	returns $1 + 0 = 1$
`Fib(4)`	returns $2 + 1 = 3$

The result is $F(4) = 3$, after $C(4) = 2F(5) - 1 = 9$ calls.

Example: execution for $n = 4$ (3/3)

Instead, `IterativeFib(4)` executes 3 iterations:

iteration	i	$next = prev + curr$	$prev$	$curr$
initialization	2	–	0	1
1	$2 \to 3$	$0 + 1 = 1$	1	1
2	$3 \to 4$	$1 + 1 = 2$	1	2
3	$4 \to 5$	$1 + 2 = 3$	2	3

The loop ends with $i = 5 > 4$ and the function returns $curr = 3$.

Summary (1/2)

The naive algorithm Fib:

- two base cases and two recursive calls; correct by strong induction
- the tree of recursive calls branches: $C(n) = 2F(n+1) - 1$ calls

Exponential growth:

- $2^{n/2} \leq C(n) \leq 2^n$ for $n \geq 2$
- $S(n) = 2C(n) = 4F(n+1) - 2 = 2^{\Theta(n)}$

Summary (2/2)

The iterative algorithm IterativeFib:

- computes each Fibonacci number only once, keeping the last two values
- $S(n) = 5n + 1 = \Theta(n)$

The lesson:

- the problem is not the recursion, but the repetition of identical subproblems
- storing and reusing their solutions is the idea of Dynamic Programming

Greatest common divisor

Introduction: Greatest common divisor

- In this section we rewrite the Euclidean algorithm, already seen in its iterative form, as a recursive algorithm
- It is an example of **tail recursion**: the recursive call is the last operation, and the algorithm can be rewritten directly as a loop
- We prove that the second argument halves every two calls, which gives a logarithmic number of calls

The problem

Given two natural numbers n, m with $n \geq 1$ and $m \geq 0$,
we consider the problem of computing
their greatest common divisor $\gcd(n, m)$
(with $\gcd(n, 0) = n$).

- In the chapter «Numerical algorithms» we proved that, for all natural numbers $n \geq 1$ and $m \geq 1$,

$$\gcd(n, m) = \gcd(m,\ n \bmod m)$$

- and we obtained the iterative algorithm `EuclideanAlgorithm`: it replaces the pair (n, m) by the pair $(m,\ n \bmod m)$ again and again, until $m = 0$

A recursive definition

- The same property, together with $\gcd(n, 0) = n$, is a recursive definition:

$$\gcd(n, m) = \begin{cases} n & \text{if } m = 0 \\ \gcd(m,\ n \bmod m) & \text{if } m \geq 1 \end{cases}$$

- This gives the recursive version of the Euclidean algorithm

Algorithm: RecursiveGCD

Input : Two natural numbers $n \geq 1$ and $m \geq 0$
Output : $\gcd(n, m)$

```
1 function RecursiveGCD(n, m)
2     if m = 0 then
3         return n                                  // base case
4     end
5     return RecursiveGCD(m, n mod m) // recursive step
6 end
```

Run in Colab: Python · C++ · Java

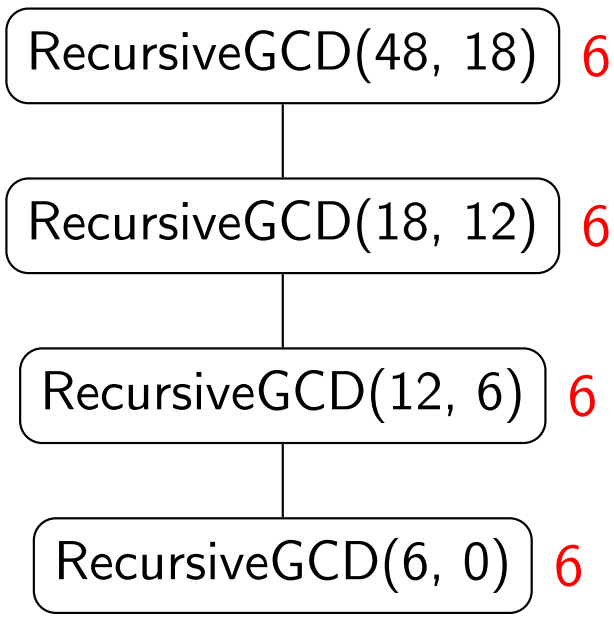
Termination of RecursiveGCD

The size of the input is the second argument m.

Since $0 \leq n \bmod m < m$,
the second argument strictly decreases at each recursive call,
hence the base case $m = 0$ is always reached.

Example: trees of recursive calls of RecursiveGCD (1/2)

Each call makes at most one recursive call, so the trees are paths.





Example: trees of recursive calls of RecursiveGCD (2/2)

- $48 \bmod 18 = 12$, $18 \bmod 12 = 6$, $12 \bmod 6 = 0$: hence $\gcd(48, 18) = 6$
- $252 \bmod 105 = 42$, $105 \bmod 42 = 21$, $42 \bmod 21 = 0$: hence $\gcd(252, 105) = 21$
- The value found by the base case is returned unchanged by all the calls

Observation 6: correctness of RecursiveGCD

For every pair of natural numbers $n \geq 1$ *and* $m \geq 0$*, the call* `RecursiveGCD`(n, m) *terminates and returns* $\gcd(n, m)$*.*

Proof

We proceed by strong induction on m.

- **Base case** ($m = 0$): the algorithm returns $n = \gcd(n, 0)$ at line 3
- **Inductive step** ($m \geq 1$): assume the claim for every second argument $k < m$ and every first argument ≥ 1
  - the algorithm calls `RecursiveGCD`(m, $n \bmod m$), where $m \geq 1$ and $n \bmod m < m$;
  - by the inductive hypothesis, this call returns $\gcd(m, n \bmod m)$;
  - this is equal to $\gcd(n, m)$ by the property $\gcd(n, m) = \gcd(m, n \bmod m)$ recalled at the beginning of this section.

□

Definition 4: tail recursion

*A recursive call is a **tail call** if it is the last operation executed by the calling function, and its result is returned unchanged. A recursive algorithm whose recursive calls are all tail calls is **tail recursive**.*

Tail recursive or not?

- `RecursiveGCD` is tail recursive: line 5 returns directly the result of the recursive call, without any further operation
- `Factorial` is **not** tail recursive:
  - after the recursive call `Factorial`$(n-1)$ returns, the result must still be multiplied by n;
  - so the frame of the caller must be kept in the stack.

From tail recursion to a loop

- A tail-recursive algorithm can always be transformed into a `while` loop:
  - the tail call is replaced by the update of the parameters;
  - the loop continues until the base case is reached.

- Applying this transformation to `RecursiveGCD`, with the update

$$(n, m) \leftarrow (m,\ \ n \bmod m),$$

  we obtain exactly the iterative `EuclideanAlgorithm`, which uses a constant amount of memory

- Many compilers apply this transformation automatically (**tail-call optimization**)

Number of steps: the second arguments

- Each call executes exactly 2 lines (lines 2–3, or lines 2 and 5), so the number of steps is twice the number of calls
- We bound the number of calls
- Let $m_0 = m$ and let $m_0, m_1, m_2, \ldots, m_K = 0$ be the second arguments of the successive calls
- The call $k \geq 1$ is `RecursiveGCD`(m_{k-1}, m_k) and therefore

$$m_{k+1} = m_{k-1} \bmod m_k, \qquad m_{k+1} < m_k.$$

Observation 7: the second argument halves every two calls

For every k such that m_{k+2} is defined,

$$m_{k+2} < \frac{m_k}{2}.$$

Proof

We have $m_{k+2} = m_k \bmod m_{k+1}$ with $m_{k+1} < m_k$. We distinguish two cases.

- If $m_{k+1} \leq m_k/2$, then

$$m_{k+2} < m_{k+1} \leq m_k/2.$$

- If $m_{k+1} > m_k/2$, then $m_k < 2m_{k+1}$, so the quotient of m_k divided by m_{k+1} is 1 and

$$m_{k+2} = m_k - m_{k+1} < m_k - m_k/2 = m_k/2.$$

□

Bounding the number of calls

- Let $m \geq 1$ and $j = \lfloor \log_2 m \rfloor + 1$, so that $2^j > m$
- Applying the Observation j times, if m_{2^j} is defined then

$$m_{2^j} < \frac{m}{2^j} < 1 \quad \Longrightarrow \quad m_{2^j} = 0.$$

- Hence $K \leq 2j$ and the number of calls $(K + 1)$ is at most $2\lfloor \log_2 m \rfloor + 3$
- Therefore

$$S(n, m) \leq 2\big(2\lfloor \log_2 m \rfloor + 3\big) = O(\log_2 m).$$

Running time of RecursiveGCD

- The input consists of the two numbers n and m, so $|I| = \Omega(\log_2 m)$ (for $m \geq 2$) and, as for `ChangeToBase2` in the chapter «Running time», $S(n, m) = O(|I|)$
- The `RecursiveGCD` algorithm makes at most $2\lfloor \log_2 m \rfloor + 3$ calls, and its worst-case running time is

$$T(|I|) = O(|I|),$$

  i.e., it is at most **linear** in the input size
- (It can be shown that the worst case occurs for two consecutive Fibonacci numbers: for example $\gcd(89, 55)$ requires 10 calls, against the bound $2\lfloor \log_2 55 \rfloor + 3 = 13$.)

Example: execution of RecursiveGCD(18, 48)

For $n = 18$ and $m = 48$ (note that $n < m$: the first call simply swaps the two numbers, since 18 mod 48 = 18):

1. `RecursiveGCD(18, 48)`: $m \neq 0$, 18 mod 48 = 18 $\Rightarrow$ calls `RecursiveGCD(48, 18)`
2. `RecursiveGCD(48, 18)`: $m \neq 0$, 48 mod 18 = 12 $\Rightarrow$ calls `RecursiveGCD(18, 12)`
3. `RecursiveGCD(18, 12)`: $m \neq 0$, 18 mod 12 = 6 $\Rightarrow$ calls `RecursiveGCD(12, 6)`
4. `RecursiveGCD(12, 6)`: $m \neq 0$, 12 mod 6 = 0 $\Rightarrow$ calls `RecursiveGCD(6, 0)`
5. `RecursiveGCD(6, 0)`: $m = 0 \Rightarrow$ base case, returns 6

The value 6 is returned unchanged by all the calls: $\gcd(18, 48) = 6$, after 5 calls.

Summary

The algorithm:

- base case $m = 0$, recursive step `RecursiveGCD`$(m,\ n \bmod m)$
- correct by strong induction on the second argument m

Tail recursion:

- the recursive call is the last operation and its result is returned unchanged
- it can be rewritten as a loop: the iterative `EuclideanAlgorithm`

Number of calls:

- the second argument halves every two calls: at most $2\lfloor \log_2 m \rfloor + 3$ calls
- $T(|I|) = O(|I|)$

Fast exponentiation

Introduction: Fast exponentiation

- In this section we compute the power x^n by **squaring**: the exponent is halved at each recursive call
- We prove the correctness by strong induction, used for $\lfloor n/2 \rfloor$ instead of $n-1$
- Unrolling the recurrence $S(n) = S(\lfloor n/2 \rfloor) + 4$ we obtain a **logarithmic** number of steps, instead of the linear number of the naive algorithm

The problem

Given a number x and a natural number n,
we consider the problem of computing the power x^n
(with $x^0 = 1$).

- The naive algorithm multiplies n times by x:

$$p := 1, \qquad p \leftarrow p \cdot x \quad (n \text{ times}),$$

which requires n multiplications and $\Theta(n)$ steps

Squaring

- A much faster algorithm is obtained by **squaring**
- Since $n = 2\lfloor n/2 \rfloor$ if n is even, and $n = 2\lfloor n/2 \rfloor + 1$ if n is odd,

$$x^n = \begin{cases} 1 & \text{if } n = 0 \\ \left(x^{\lfloor n/2 \rfloor}\right)^2 & \text{if } n \geq 1 \text{ is even} \\ x \cdot \left(x^{\lfloor n/2 \rfloor}\right)^2 & \text{if } n \text{ is odd} \end{cases}$$

  (for n odd, $\lfloor n/2 \rfloor = (n-1)/2$)

- The size of the exponent is halved at each recursive call

Algorithm: FastPower

Input : A number x and a natural number n
Output : The power x^n

```
1  function FastPower(x, n)
2      if n = 0 then
3          return 1                                  // base case
4      end
5      y := FastPower(x, ⌊n/2⌋)          // one recursive call
6      if n mod 2 = 0 then
7          return y · y                                // n even
8      else
9          return x · y · y                             // n odd
10     end
11 end
```

Run in Colab: Python · C++ · Java

One recursive call, used twice

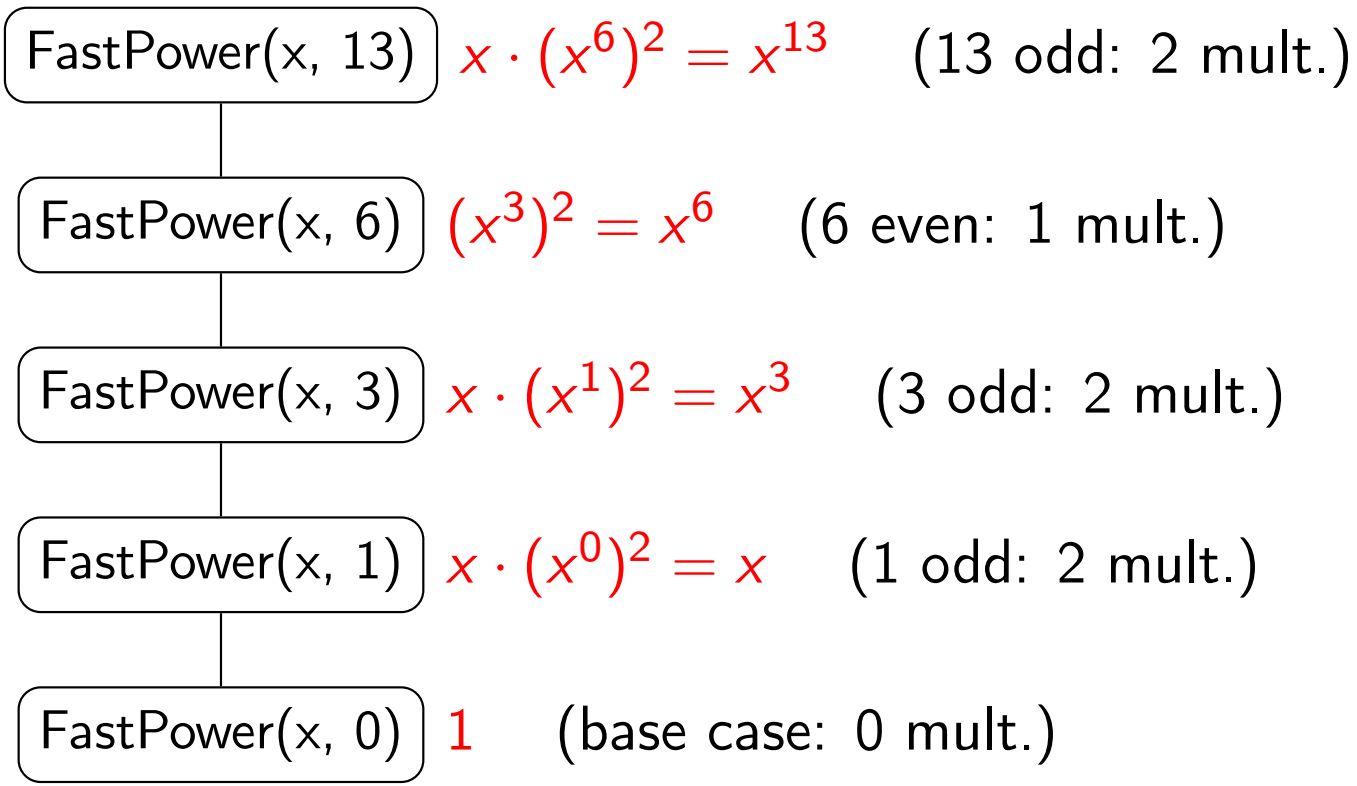
- The result of the recursive call is stored in the variable y and used twice
- Writing

$$\texttt{FastPower}(x, \lfloor n/2 \rfloor) \cdot \texttt{FastPower}(x, \lfloor n/2 \rfloor)$$

  would make **two** identical recursive calls and would destroy the efficiency of the algorithm
- This variant is analysed in the exercises of the chapter

Example: tree of recursive calls of FastPower(x, 13) (1/2)

The exponents of the calls are $13, 6, 3, 1, 0$ ($\lfloor 13/2 \rfloor = 6$, $\lfloor 6/2 \rfloor = 3$, $\lfloor 3/2 \rfloor = 1$, $\lfloor 1/2 \rfloor = 0$). On the right of each call: the value returned and the number of multiplications executed by that call.



Example: tree of recursive calls of FastPower(x, 13) (2/2)

- Total number of multiplications: $2 + 1 + 2 + 2 + 0 = 7$, against the 13 multiplications of the naive algorithm
- Note that $13 = (1101)_2$:
  - the odd/even exponents met going up from $\texttt{FastPower}(x, 1)$ to $\texttt{FastPower}(x, 13)$ are odd, odd, even, odd;
  - they are exactly the binary digits $1, 1, 0, 1$ of 13 read from the most significant one.
- In general, the number of multiplications is the number of binary digits of n plus the number of digits equal to 1: here $4 + 3 = 7$

Observation 8: correctness of FastPower

For every number x and every natural number n, the call `FastPower`(x, n) *terminates and returns* x^n.

Proof

We proceed by strong induction on n.

- **Base case** ($n = 0$): the algorithm returns $1 = x^0$ at line 3
- **Inductive step** ($n \geq 1$): assume that `FastPower`(x, k) returns x^k for every $k < n$. Since $n \geq 1$, we have $\lfloor n/2 \rfloor < n$, so by the inductive hypothesis line 5 assigns $y = x^{\lfloor n/2 \rfloor}$
  - if n is even, then $n = 2\lfloor n/2 \rfloor$ and line 7 returns $y \cdot y = x^{2\lfloor n/2 \rfloor} = x^n$;
  - if n is odd, then $n = 2\lfloor n/2 \rfloor + 1$ and line 9 returns $x \cdot y \cdot y = x^{2\lfloor n/2 \rfloor + 1} = x^n$.

Note that strong induction is needed: the inductive hypothesis is used for $k = \lfloor n/2 \rfloor$, not for $k = n - 1$. □

Number of steps: the recurrence

- Let $S(n)$ denote the number of steps of `FastPower` on the exponent n
- If $n = 0$, lines 2–3 are executed:

$$S(0) = 2.$$

- If $n \geq 1$, lines 2, 5, 6 and one of the lines 7, 9 are executed once each, plus the steps of the call on $\lfloor n/2 \rfloor$:

$$S(n) = S(\lfloor n/2 \rfloor) + 4 \qquad (n \geq 1).$$

Number of steps: unrolling the recurrence (1/2)

We solve the recurrence by unrolling it. Since $\lfloor \lfloor n/2^k \rfloor /2 \rfloor = \lfloor n/2^{k+1} \rfloor$,

$$\begin{aligned} S(n) &= S(\lfloor n/2 \rfloor) + 4 \\ &= S(\lfloor n/4 \rfloor) + 2 \cdot 4 \\ &\vdots \\ &= S(\lfloor n/2^k \rfloor) + k \cdot 4, \end{aligned}$$

as long as $\lfloor n/2^{k-1} \rfloor \geq 1$.

Number of steps: unrolling the recurrence (2/2)

- Now $\lfloor n/2^k \rfloor \geq 1$ if and only if $2^k \leq n$, i.e., $k \leq \lfloor \log_2 n \rfloor$
- Hence, for $n \geq 1$, the exponents

$$n, \lfloor n/2 \rfloor, \ldots, \lfloor n/2^{\lfloor \log_2 n \rfloor} \rfloor = 1$$

  are all ≥ 1, and the next one is 0
- Choosing $k = \lfloor \log_2 n \rfloor + 1$:

$$S(n) = S(0) + 4\big(\lfloor \log_2 n \rfloor + 1\big) = 4\lfloor \log_2 n \rfloor + 6.$$

Steps, calls and multiplications

- For example, $S(13) = 4 \cdot 3 + 6 = 18$
- As shown for `ChangeToBase2` in the chapter «Running time»,

$$\lim_{n \to \infty} \frac{4\lfloor \log_2 n \rfloor + 6}{\log_2 n} = 4 \in \mathbb{R}_{>0},$$

  hence $S(n) = \Theta(\log_2 n)$
- For $n \geq 1$, the number of calls, and therefore the depth of the recursion, is $\lfloor \log_2 n \rfloor + 2$ (the number of binary digits of n, plus the call with exponent 0)
- The number of multiplications is at most $2(\lfloor \log_2 n \rfloor + 1)$, against n for the naive algorithm
- For example, for $n = 1000 = (1111101000)_2$: $10 + 6 = 16$ multiplications instead of 1000

Running time of FastPower

- Assume that x and all the intermediate results use a constant number of bits (so each multiplication is one step)
- The input size is $|I| = \Theta(\log_2 n)$, hence, as for `ChangeToBase2` in the chapter «Running time», $S(n) = \Theta(\log_2 n)$ gives $T(|I|) = \Theta(|I|)$

The number of steps of the `FastPower` algorithm is

$$S(n) = 4\lfloor \log_2 n \rfloor + 6 = \Theta(\log_2 n),$$

and its worst-case running time is

$$T(|I|) = \Theta(|I|),$$

i.e., it is **linear** in the input size. The naive algorithm executes $\Theta(n)$ steps, i.e., it is **exponential** in the input size.

Example: execution of FastPower(3, 13) (1/2)

For $x = 3$ and $n = 13$.

Calls

1. `FastPower(3, 13)`: $13 \neq 0 \Rightarrow$ calls `FastPower(3, 6)` ($\lfloor 13/2 \rfloor = 6$)
2. `FastPower(3, 6)`: $6 \neq 0 \Rightarrow$ calls `FastPower(3, 3)` ($\lfloor 6/2 \rfloor = 3$)
3. `FastPower(3, 3)`: $3 \neq 0 \Rightarrow$ calls `FastPower(3, 1)` ($\lfloor 3/2 \rfloor = 1$)
4. `FastPower(3, 1)`: $1 \neq 0 \Rightarrow$ calls `FastPower(3, 0)` ($\lfloor 1/2 \rfloor = 0$)
5. `FastPower(3, 0)`: base case, returns 1

Example: execution of FastPower(3, 13) (2/2)

Returns

1. `FastPower(3, 1)`: $y = 1$, 1 is odd $\Rightarrow$ returns $3 \cdot 1 \cdot 1 = 3$
2. `FastPower(3, 3)`: $y = 3$, 3 is odd $\Rightarrow$ returns $3 \cdot 3 \cdot 3 = 27$
3. `FastPower(3, 6)`: $y = 27$, 6 is even $\Rightarrow$ returns $27 \cdot 27 = 729$
4. `FastPower(3, 13)`: $y = 729$, 13 is odd $\Rightarrow$ returns $3 \cdot 729 \cdot 729 = 1594323$

The algorithm terminates with result $1594323 = 3^{13}$, after 5 calls and 7 multiplications.

Summary

The algorithm:

- one recursive call on $\lfloor n/2 \rfloor$, whose result y is used twice
- correct by strong induction (hypothesis used for $\lfloor n/2 \rfloor$)

Number of steps:

- $S(n) = S(\lfloor n/2 \rfloor) + 4$, hence $S(n) = 4\lfloor \log_2 n \rfloor + 6 = \Theta(\log_2 n)$
- depth $\lfloor \log_2 n \rfloor + 2$; at most $2(\lfloor \log_2 n \rfloor + 1)$ multiplications

Running time:

- $T(|I|) = \Theta(|I|)$, against the exponential naive algorithm

Towers of Hanoi

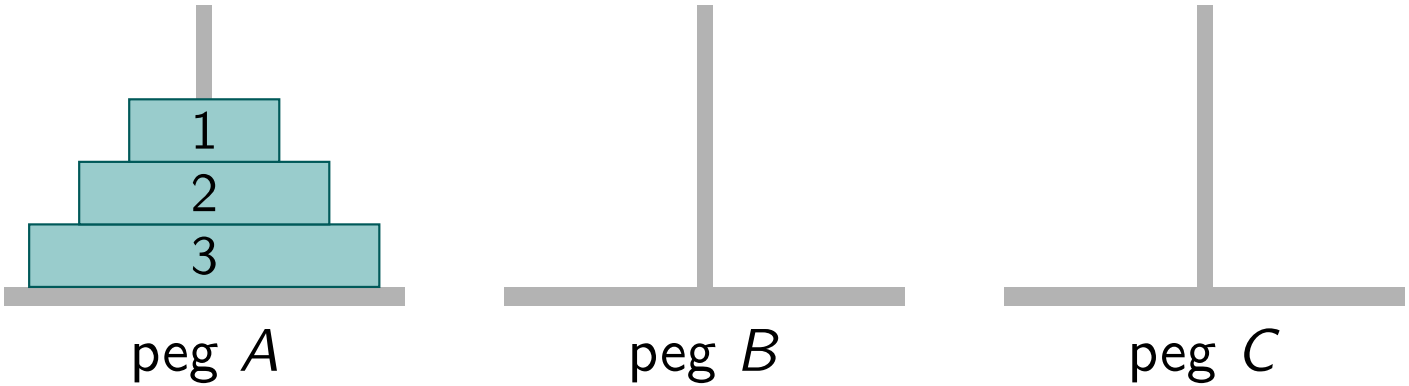
Introduction: Towers of Hanoi

- In this section we solve a classical puzzle with a recursive algorithm that makes **two** recursive calls on $n - 1$ disks
- We draw its tree of recursive calls and prove that it executes $2^n - 1$ moves
- Finally we show that no algorithm can do better: the exponential number of moves is **optimal**

The problem

- There are three pegs A, B, C and $n \geq 1$ disks of different sizes, numbered $1, 2, \ldots, n$ from the smallest to the largest
- Initially all the disks are stacked on peg A, in decreasing order of size from the bottom to the top
- We consider the problem of moving all the disks to peg C, with the following rules:
  - only one disk is moved at a time, taking the top disk of a peg and putting it on top of another peg;
  - a disk can never be placed on top of a smaller disk.

The initial configuration



All the disks must be moved from peg A to peg C,
one at a time, never placing a disk on top of a smaller one.

Recursive idea

To move n disks from peg A to peg C using peg B as auxiliary peg:

1. move the $n-1$ smallest disks from A to B (using C as auxiliary peg), recursively;
2. move the largest disk n from A to C;
3. move the $n-1$ smallest disks from B to C (using A as auxiliary peg), recursively.

- During steps 1 and 3, disk n is at the bottom of a peg and it is larger than all the moved disks, so it never violates the rules
- The $n-1$ disks can be moved as if disk n were not there

Algorithm: Hanoi

Input : The number of disks $n \geq 1$, the source peg *from*, the destination peg *to*, the auxiliary peg *aux*

Output : The sequence of moves that transfers disks $1, 2, \ldots, n$ from *from* to *to*

```
1 function Hanoi(n, from, to, aux)
2     if n = 1 then
3         move disk 1 from from to to                    // base case
4     else
5         Hanoi(n − 1, from, aux, to)
6         move disk n from from to to
7         Hanoi(n − 1, aux, to, from)
8     end
9 end
```

The initial call is `Hanoi`(n, A, C, B).

Run in Colab: Python · C++ · Java

Observation 9: correctness of Hanoi

For every natural number $n \geq 1$, if disks $1, 2, \ldots, n$ are the top n disks of peg from and all the disks on pegs to and aux are larger than n, then `Hanoi`$(n, from, to, aux)$ *moves disks $1, 2, \ldots, n$ on top of peg to, respecting the rules.*

Proof

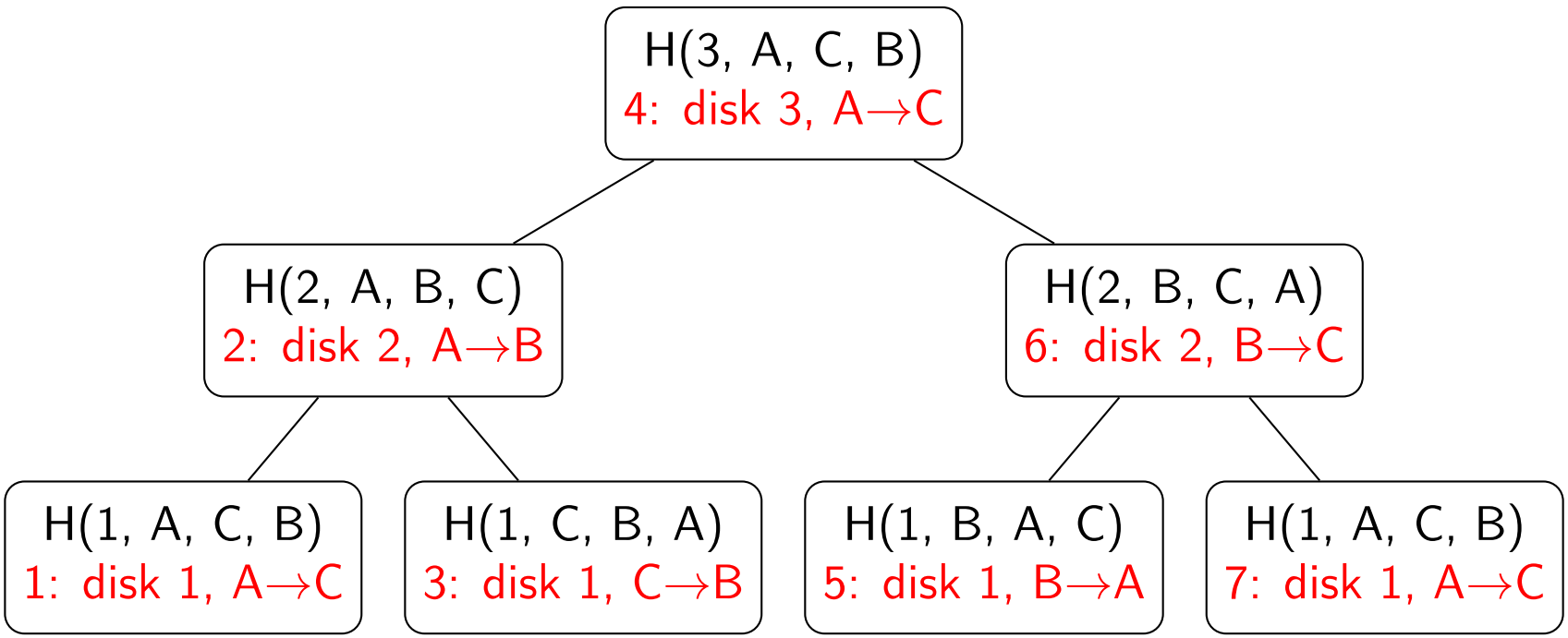
We proceed by induction on n.

- **Base case** ($n = 1$): the single move of disk 1 is valid, since disk 1 is the smallest disk
- **Inductive step** ($n \geq 2$):
  - by the inductive hypothesis, the call of line 5 correctly moves the tower of the $n - 1$ smallest disks from *from* to *aux* (disk n and the disks below it are larger, so they do not interfere);
  - then disk n is on top of peg *from*, and every disk on peg *to* is larger than n: line 6 moves disk n correctly;
  - finally, by the inductive hypothesis, the call of line 7 moves the $n - 1$ smallest disks from *aux* on top of disk n.

□

Example: tree of recursive calls of Hanoi(3, A, C, B) (1/2)

Each node shows the call (with arguments *n*, *from*, *to*, *aux*) and, in red, the move executed by the call itself (line 3 or line 6), numbered in the order of execution.



Example: tree of recursive calls of Hanoi(3, A, C, B) (2/2)

The moves are executed in the order of an **in-order** visit of the tree (left subtree, node, right subtree):

move	disk	from	to
1	1	A	C
2	2	A	B
3	1	C	B
4	3	A	C
5	1	B	A
6	2	B	C
7	1	A	C

The tree has 7 nodes and every node executes exactly one move: 7 moves in total.

Number of moves

- Let $M(n)$ be the number of moves executed by `Hanoi` with n disks. From the pseudo-code,

$$M(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2M(n-1) + 1 & \text{if } n \geq 2 \end{cases}$$

Number of moves: unrolling

Unrolling:

$$\begin{aligned} M(n) &= 2M(n-1) + 1 \\ &= 4M(n-2) + 2 + 1 \\ &= 8M(n-3) + 4 + 2 + 1 \\ &\vdots \\ &= 2^{n-1}M(1) + 2^{n-2} + \cdots + 2 + 1 = 2^n - 1. \end{aligned}$$

We prove it formally.

Observation 10: number of moves of Hanoi

For every natural number $n \geq 1$,

$$M(n) = 2^n - 1.$$

Proof

We proceed by induction on n.

- **Base case** ($n = 1$): $M(1) = 1 = 2^1 - 1$
- **Inductive step** ($n \geq 2$): assume $M(n-1) = 2^{n-1} - 1$. Then

$$\begin{aligned} M(n) &= 2M(n-1) + 1 \\ &= 2\left(2^{n-1} - 1\right) + 1 \\ &= 2^n - 2 + 1 = 2^n - 1. \end{aligned}$$

□

Number of steps and depth

- The number of steps satisfies
  - $S(1) = 2$ (lines 2–3);
  - $S(n) = 2S(n-1) + 4$ for $n \geq 2$ (lines 2, 5, 6, 7, plus the two recursive calls).
- Unrolling as above,

$$\begin{aligned} S(n) &= 2^{n-1}S(1) + 4\left(2^{n-2} + \cdots + 2 + 1\right) \\ &= 2^n + 4\left(2^{n-1} - 1\right) = 3 \cdot 2^n - 4 = \Theta(2^n), \end{aligned}$$

  since $\lim_{n\to\infty}(3 \cdot 2^n - 4)/2^n = 3 \in \mathbb{R}_{>0}$
- The depth of the recursion is only n: the memory used by the call stack is $\Theta(n)$

Optimality

The exponential number of moves
is not a weakness of the algorithm:

no algorithm can solve the problem
with fewer moves.

Observation 11: optimality of Hanoi

Let $M^*(n)$ be the minimum number of moves needed to transfer a tower of $n \geq 1$ disks from one peg to another one. Then

$$M^*(n) = 2^n - 1.$$

Sketch of proof (1/3)

- Since the algorithm `Hanoi` uses $2^n - 1$ moves, $M^*(n) \leq 2^n - 1$
- We show $M^*(n) \geq 2^n - 1$ by induction on n
- For $n = 1$, at least one move is needed
- Let $n \geq 2$. The largest disk n must be moved **at least once**

Sketch of proof (2/3)

- Consider the **first** move of disk n, from the source peg to some other peg P:
  - at that moment disk n is the only disk on the source peg and peg P is empty;
  - so the $n-1$ smaller disks are all stacked on the third peg;
  - moving them there from the source peg required at least $M^*(n-1)$ moves.

- Consider now the **last** move of disk n, onto the destination peg:
  - after this move, the $n-1$ smaller disks are all stacked on a peg different from the destination;
  - they must still be transferred on top of disk n: this requires at least $M^*(n-1)$ further moves.

Sketch of proof (3/3)

These two groups of moves are distinct (one happens before the first move of disk n, the other after its last move), hence

$$\begin{aligned} M^*(n) &\geq M^*(n-1) + 1 + M^*(n-1) \\ &= 2M^*(n-1) + 1 \\ &\geq 2\left(2^{n-1} - 1\right) + 1 = 2^n - 1. \end{aligned}$$

□

Running time of Hanoi

- The `Hanoi` algorithm executes $M(n) = 2^n - 1$ moves and $S(n) = 3 \cdot 2^n - 4 = \Theta(2^n)$ steps, with depth of recursion n
- The number of moves is **optimal**: every algorithm must output $2^n - 1$ moves
- So the running time of any algorithm for this problem is exponential in n (and therefore doubly exponential in the input size $|I| = \Theta(\log_2 n)$)
- With $n = 64$ disks (the original legend) and one move per second, $2^{64} - 1$ seconds are more than $5.8 \cdot 10^{11}$ years

Example: execution of Hanoi(3, A, C, B) (1/2)

Indentation = depth in the stack; the printed lines are the moves of the tree above.

call	action
`Hanoi(3,A,C,B)`	calls `Hanoi(2,A,B,C)`
`  Hanoi(2,A,B,C)`	calls `Hanoi(1,A,C,B)`
`    Hanoi(1,A,C,B)`	prints **move disk 1 from A to C**, returns
`  Hanoi(2,A,B,C)`	prints **move disk 2 from A to B**, calls `Hanoi(1,C,B,A)`
`    Hanoi(1,C,B,A)`	prints **move disk 1 from C to B**, returns
`  Hanoi(2,A,B,C)`	returns
`Hanoi(3,A,C,B)`	prints **move disk 3 from A to C**, calls `Hanoi(2,B,C,A)`

Example: execution of Hanoi(3, A, C, B) (2/2)

call	action
`Hanoi(2,B,C,A)`	calls `Hanoi(1,B,A,C)`
`Hanoi(1,B,A,C)`	prints **move disk 1 from B to A**, returns
`Hanoi(2,B,C,A)`	prints **move disk 2 from B to C**, calls `Hanoi(1,A,C,B)`
`Hanoi(1,A,C,B)`	prints **move disk 1 from A to C**, returns
`Hanoi(2,B,C,A)`	returns
`Hanoi(3,A,C,B)`	returns

All the 7 moves are printed and all the disks are on peg C.

Summary

The algorithm:

- move $n-1$ disks to the auxiliary peg, move disk n, move the $n-1$ disks back on top of it
- correct by induction on n; the moves follow an in-order visit of the tree

Number of moves and steps:

- $M(n) = 2M(n-1) + 1 = 2^n - 1$; $S(n) = 3 \cdot 2^n - 4 = \Theta(2^n)$; depth n

Optimality:

- every algorithm needs at least $2^n - 1$ moves: the problem itself is exponential

Summary: the five algorithms (1/3)

algorithm	recurrence for $S(n)$
`Factorial`	$S(n) = S(n-1) + 2$
`Fib` (naive)	$S(n) = S(n-1) + S(n-2) + 2$
`RecursiveGCD`	$m_{k+2} < m_k/2$
`FastPower`	$S(n) = S(\lfloor n/2 \rfloor) + 4$
`Hanoi`	$S(n) = 2S(n-1) + 4$

Summary: the five algorithms (2/3)

algorithm	$S(n)$
`Factorial`	$2n + 2 = \Theta(n)$
`Fib` (naive)	$4F(n+1) - 2 = 2^{\Theta(n)}$
`RecursiveGCD`	$O(\log_2 m)$
`FastPower`	$4\lfloor \log_2 n \rfloor + 6 = \Theta(\log_2 n)$
`Hanoi`	$3 \cdot 2^n - 4 = \Theta(2^n)$

Summary: the five algorithms (3/3)

algorithm	calls	depth
`Factorial`	$n+1$	$n+1$
`Fib` (naive)	$2F(n+1)-1$	$\max(n,1)$
`RecursiveGCD`	$\leq 2\lfloor \log_2 m \rfloor + 3$	$=$ calls
`FastPower`	$\lfloor \log_2 n \rfloor + 2$	$=$ calls
`Hanoi`	$2^n - 1$	n

Summary: key points of the chapter (1/2)

- A **recursive algorithm** solves the **base cases** directly and reduces every other instance to **strictly smaller** instances of the same problem; this guarantees termination
- The execution is described by the **tree of recursive calls**: the number of steps depends on the **number of nodes**, the memory used by the **call stack** on the **depth** of the tree
- Correctness is proved by **(strong) induction** on the size of the input: base cases $\leftrightarrow$ base of the induction, recursive step $\leftrightarrow$ inductive step (as initialization and maintenance for loop invariants)

Summary: key points of the chapter (2/2)

- The number of steps is described by a **recurrence** for $S(n)$:
  - decreasing by one with constant work gives $\Theta(n)$ (`Factorial`);
  - halving with constant work gives $\Theta(\log_2 n)$ (`FastPower`);
  - two calls on $n-1$ give $\Theta(2^n)$ (`Hanoi`).
- Recursion can be very inefficient when the **same subproblems** are solved many times (naive `Fib`): this motivates **Dynamic Programming**
- A **tail-recursive** algorithm (`RecursiveGCD`) can be directly rewritten as a loop